



COMPUTER
ENGINEERING

QUEZON CITY

COURSE CODE/TITLE: CPE 010 - Data Structures and Algorithms

SECTION: CPE21S1

DATE PERFORMED: Sep 10, 2026

DATE SUBMITTED: Sep 15, 2026

NAME: Benj Federic De Leon

INSTRUCTOR: Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

SortingAlgo.h

```
#ifndef SORTALGORITHMS_H
#define SORTALGORITHMS_H

#include <utility> // std::swap

//A.1 SHELL SORT
void shellSort(int arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        //gap-insertion sort for this interval
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}

//A.2 MERGE SORT
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;    // size of left sub-array
    int n2 = right - mid;       // size of right sub-array
```

```

    int* L = new int[n1];
    int* R = new int[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    delete[] L;
    delete[] R;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

//A.3 QUICK SORT
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;

```

```

        std::swap(arr[i], arr[j]);
    }

    }

    std::swap(arr[i + 1], arr[high]); // place pivot in its correct
spot
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

#endif

```

Code Analysis:

This C++ header implements three sorting algorithms: Shell Sort improves insertion sort by swapping distant elements using a shrinking gap; Merge Sort splits the array into halves and recursively merges them using temporary.

```

#include <iostream>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo.h"

int main() {
    //Preparation Task: generate 100 unsorted random elements
    int original[SIZE];
    srand(static_cast<unsigned int>(time(0)));
    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 100;
    }

    std::cout << "=== Table 8-1: Original Unsorted Array ===" <<

```

```

std::endl;
    printArray(original, SIZE);

    return 0;
}

```

Table 8-1. Array of Values for Sort Algorithm Testing

A.1 Shell Sort

Ap1_shellSort.cpp

```

#include <iostream>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo.h"

const int SIZE = 100;

void printArray(int arr[], int n);

void copyArray(int src[], int dest[], int n);

int main() {
    //Preparation Task: generate 100 unsorted random elements
    int original[SIZE];
    srand(static_cast<unsigned int>(time(0)));
    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 100;
    }

    std::cout << "=== Table 8-1: Original Unsorted Array ===" <<
std::endl;
    printArray(original, SIZE);

    //A.1 Shell Sort
    int shellArr[SIZE];
    copyArray(original, shellArr, SIZE);

```

```

    shellSort(shellArr, SIZE);
    std::cout << "=== Table 8-2: Shell Sort Result ===" << std::endl;
    printArray(shellArr, SIZE);

    return 0;
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl << std::endl;
}

void copyArray(int src[], int dest[], int n) {
    for (int i = 0; i < n; i++) dest[i] = src[i];
}

```

Output

```

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA8p1> cd "c:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA8p1\" ; if ($?) { g++ Ap1_shellSort.cpp -o Ap1_shellSort } ; if ($?) { .\Ap1_shellSort }
=== Table 8-1: Original Unsorted Array ===
80 80 13 58 61 70 7 70 64 91 92 37 97 18 30 50 39 98 95 96 93 18 96 1 5 16 97 83 17 51 70 58 66 89 0 74 53 84 39 10 39 31 74 18 24 20 34
1 39 53 52 42 85 63 26 55 72 31 46 50 82 92 11 67 78 83 44 63 16 46 72 64 51 78 17 69 76 98 11 79 13 25 38 87 29 3 75 38 82 73 20 11 95 1
3 11 64 76 72 51 68

=== Table 8-2: Shell Sort Result ===
0 1 1 3 5 7 10 11 11 11 11 13 13 16 16 17 17 18 18 18 20 20 24 25 26 29 30 31 31 34 37 38 38 39 39 39 39 42 44 46 46 50 51 51 51 52
53 53 55 58 58 61 63 63 64 64 66 67 68 69 70 70 70 72 72 72 73 74 74 75 76 76 78 78 79 80 80 82 82 83 83 84 85 87 89 91 92 92 93 95 9
5 96 96 97 97 98 98

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA8p1>

```

Table 8-2. Shell Sort Technique

Code Analysis:

This C++ program tests Shell Sort by generating 100 random integers (0–99), printing the initial unsorted array, copying it to prevent modifying the original data, running shellSort, and displaying the final sorted array..

A.2 Merge Sort

Ap2_mergeSort.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo.h"

const int SIZE = 100;

void printArray(int arr[], int n);

void copyArray(int src[], int dest[], int n);

int main() {
    //Preparation Task: generate 100 unsorted random elements
    int original[SIZE];
    srand(static_cast<unsigned int>(time(0)));
    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 100;
    }

    std::cout << "=== Original Unsorted Array ===" << std::endl;
    printArray(original, SIZE);

    //A.2 Merge Sort
    int mergeArr[SIZE];
    copyArray(original, mergeArr, SIZE);
    mergeSort(mergeArr, 0, SIZE - 1);
    std::cout << "=== Table 8-3: Merge Sort Result ===" <<
```

```

std::endl;
    printArray(mergeArr, SIZE);

    return 0;
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl << std::endl;
}

void copyArray(int src[], int dest[], int n) {
    for (int i = 0; i < n; i++) dest[i] = src[i];
}

```

Output

```

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA8p1> cd "c:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA8p1\" ; if ($?) { g++ Ap2_mergeSort.cpp -o Ap2_mergeSort } ; if ($?) { .\Ap2_mergeSort }
=== Original Unsorted Array ===
76 38 23 34 12 40 47 8 89 57 79 32 2 24 35 0 42 84 20 34 46 25 90 76 33 15 64 51 19 21 61 79 31 1 28 72 47 76 22 63 53 96 94 0 83 92 72
18 28 82 76 56 46 36 71 19 96 70 72 5 67 23 36 89 95 64 36 90 68 78 53 79 13 0 85 51 42 8 35 78 43 48 33 67 23 88 74 43 7 70 14 53 76 4
86 16 58 14 99

=== Table 8-3: Merge Sort Result ===
0 0 0 1 2 5 7 7 8 8 12 13 14 14 15 16 18 19 19 20 21 22 23 23 23 24 25 28 28 31 32 33 33 34 34 35 35 36 36 36 38 40 41 42 42 43 43 46 46
47 47 48 51 51 53 53 53 56 57 58 61 63 64 64 67 67 68 70 70 71 72 72 72 74 76 76 76 76 78 78 79 79 79 82 83 84 85 86 88 89 89 90 90 9
94 95 96 96 99

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA8p1>

```

Table 8-3. Merge Sort Algorithm

Code Analysis:

This C++ program tests Merge Sort by generating 100 random integers (0–99), printing the unsorted array,

copying it to a target array, running mergeSort(mergeArr, 0, SIZE - 1) with valid start and end indices, and displaying the sorted result.

A.3 Quick Sort

Ap3_quickSort.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo.h"

const int SIZE = 100;

void printArray(int arr[], int n);

void copyArray(int src[], int dest[], int n);

int main() {
    //Preparation Task: generate 100 unsorted random elements
    int original[SIZE];
    srand(static_cast<unsigned int>(time(0)));
    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 100;
    }

    std::cout << "=== Table 8-1: Original Unsorted Array ===" <<
std::endl;
    printArray(original, SIZE);

    ///A.3 Quick Sort
    int quickArr[SIZE];
    copyArray(original, quickArr, SIZE);
    quickSort(quickArr, 0, SIZE - 1);
    std::cout << "=== Table 8-4: Quick Sort Result ===" << std::endl;
```

```

        printArray(quickArr, SIZE);

        return 0;
    }

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl << std::endl;
}

void copyArray(int src[], int dest[], int n) {
    for (int i = 0; i < n; i++) dest[i] = src[i];
}

```

Output

```

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA8p1> cd "c:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CP
E21S1\HoA8p1\"; if ($?) { g++ Ap3_quickSort.cpp -o Ap3_quickSort }; if ($?) { .\Ap3_quickSort }
=== Table 8-1: Original Unsorted Array ===
79 79 90 31 24 22 11 25 0 93 8 63 56 88 93 17 90 36 21 72 11 71 5 97 81 32 74 36 91 11 12 37 19 53 49 63 99 43 9 83 3 47 75 43 53 84 63 9
6 78 47 2 65 68 25 19 57 97 27 43 99 63 16 76 31 44 34 37 23 25 48 5 72 85 98 13 47 83 57 27 4 11 29 16 92 0 44 88 24 88 80 97 29 64 92 5
3 38 92 7 24 98

=== Table 8-4: Quick Sort Result ===
0 0 2 3 4 5 5 7 8 9 11 11 11 11 12 13 16 16 17 19 19 21 22 23 24 24 24 25 25 25 27 27 29 29 31 31 32 34 36 36 37 37 38 43 43 43 44 44 47
47 47 48 49 53 53 53 56 57 57 63 63 63 63 64 65 68 71 72 72 74 75 76 78 79 79 80 81 83 83 84 85 88 88 88 90 90 91 92 92 92 93 93 96 97 97
97 98 98 99 99

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\HoA8p1>

```

Table 8-4. Quick Sort Algorithm

Code Analysis:

This C++ program tests Quick Sort by generating 100 random integers (0–99), printing the initial unsorted array, copying it to a separate array to preserve original data, running quickSort(quickArr, 0, SIZE - 1) passing bounds 0 and 99, and displaying the sorted output.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have $O(N \cdot \log N)$ for their time complexity?

supplementary.cpp

supplementary.cpp > Problem10

```
1  #include <iostream>
2  #include <ctime>
3  #include "SortingAlgo.h"
4
5  void printArr(int arr[], int n) {
6      for (int i = 0; i < n; i++) {
7          std::cout << arr[i] << " ";
8      }
9      std::cout << std::endl;
10 }
11
12 //PROBLEM 1
13 void QuickShellSort(int arr[], int low, int high) {
14     if (low < high) {
15         int pivotIndex = partition(arr, low, high);
16
17         int leftSize = pivotIndex - low;
18         int rightSize = high - pivotIndex;
19
20         //Sort left sub-list with shellSort instead of recursive quicksort
21         if (leftSize > 0) {
22             int* leftSub = new int[leftSize];
23             for (int i = 0; i < leftSize; i++)
24                 leftSub[i] = arr[low + i];
25
26             shellSort(leftSub, leftSize);
27
28             for (int i = 0; i < leftSize; i++)
29                 arr[low + i] = leftSub[i];
30
31             delete[] leftSub;
32         }
33
34         //Sort right sub-list with shellSort instead of recursive quicksort
35         if (rightSize > 0) {
36             int* rightSub = new int[rightSize];
37             for (int i = 0; i < rightSize; i++)
38                 rightSub[i] = arr[pivotIndex + 1 + i];
39
40             shellSort(rightSub, rightSize);
41
42             for (int i = 0; i < rightSize; i++)
43                 arr[pivotIndex + 1 + i] = rightSub[i];
44         }
45     }
46 }
```

```

44         delete[] rightSub;
45     }
46 }
47
48 }
49
50 void Problem1() {
51     std::cout << "Problem 1: Quicksort" << std::endl;
52
53     int arr[] = {8, 24, 7, 6, 18, 20, 30};
54     int n = sizeof(arr) / sizeof(arr[0]);
55
56     std::cout << "Original array: ";
57     printArr(arr, n);
58
59     QuickShellSort(arr, 0, n - 1);
60
61     std::cout << "Sorted array (sorted by shellSort): ";
62     printArr(arr, n);
63     std::cout << std::endl;
64 }
65
66 //PROBLEM 2
67 void Problem2() {
68     std::cout << "Problem 2: Quicksort vs Merge Sort" << std::endl;
69
70     int original[] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};
71     int n = sizeof(original) / sizeof(original[0]);
72
73     std::cout << "Original array: ";
74     printArr(original, n);
75
76     //Quick Sort
77     int quickArr[15];
78     for (int i = 0; i < n; i++) quickArr[i] = original[i];
79
80     clock_t startQ = clock();
81     quickSort(quickArr, 0, n - 1);
82     clock_t endQ = clock();

```

```

83
84     std::cout << "Quicksort result: ";
85     printArr(quickArr, n);
86     std::cout << "Quicksort time: " << (endQ - startQ) << std::endl;
87
88     //Merge Sort
89     int mergeArr[15];
90     for (int i = 0; i < n; i++) mergeArr[i] = original[i];
91
92     clock_t startM = clock();
93     mergeSort(mergeArr, 0, n - 1);
94     clock_t endM = clock();
95
96     std::cout << "Merge sort result: ";
97     printArr(mergeArr, n);
98     std::cout << "Merge sort time: " << (endM - startM) << std::endl;
99 }
100
101 int main() {
102
103     Problem1();
104     Problem2();
105
106     return 0;
107 }

```

Output:

```

PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-CPE21S1\
E21S1\HoA8p1\" ; if ($?) { g++ supplementary.cpp -o supplementary }
Problem 1: Quicksort
Original array: 8 24 7 6 18 20 30
Sorted array (sorted by shellSort): 6 7 8 18 20 24 30

Problem 2: Quicksort vs Merge Sort
Original array: 4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
Quicksort result: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
Quicksort time: 0
Merge sort result: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
Merge sort time: 0

```

Code Analysis:

This code demonstrates a sorting method by combining Quick Sort partitioning with dynamic Shell Sort sub-array sorting, alongside an empirical timer comparing Quick Sort and Merge Sort performance.

Conclusion:

In conclusion, each sorting algorithm has its own strengths: Shell Sort uses minimal memory, Merge Sort delivers consistent speed, and Quick Sort is usually the fastest overall. The main learning is that mixing algorithms works best when done directly on the same array without creating temporary copies, and measuring speed requires large data samples to see real results.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, Guide, Ideas and Error Checking.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Was used as a guide for understanding the problem of the task.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.